Analysis and Results

Insurance Portfolio Analysis

Part 1 — Total Premium Collected (2024)

SQL Query

```
SELECT
      SUM(Premium) AS Total_Premium_Collected_2024
FROM
      policy_sales_data
WHERE
      strftime('%Y', Policy_Purchase_Date) = '2024';
```

Result

Total Premium Collected = ₹240,110,800

```
query = """
SELECT
SUM(Premium) AS Total_Premium_Collected_2024
FROM policy_sales_data
WHERE strftime('%Y', Policy_Purchase_Date) = '2024';
"""

pd.read_sql_query(query, conn)
```

	Total_Premium_Collected_2024
0	240110800

Interpretation

This represents the total premium revenue generated from all policies sold in 2024.

Part 2 — Monthly Claim Cost

SQL Query

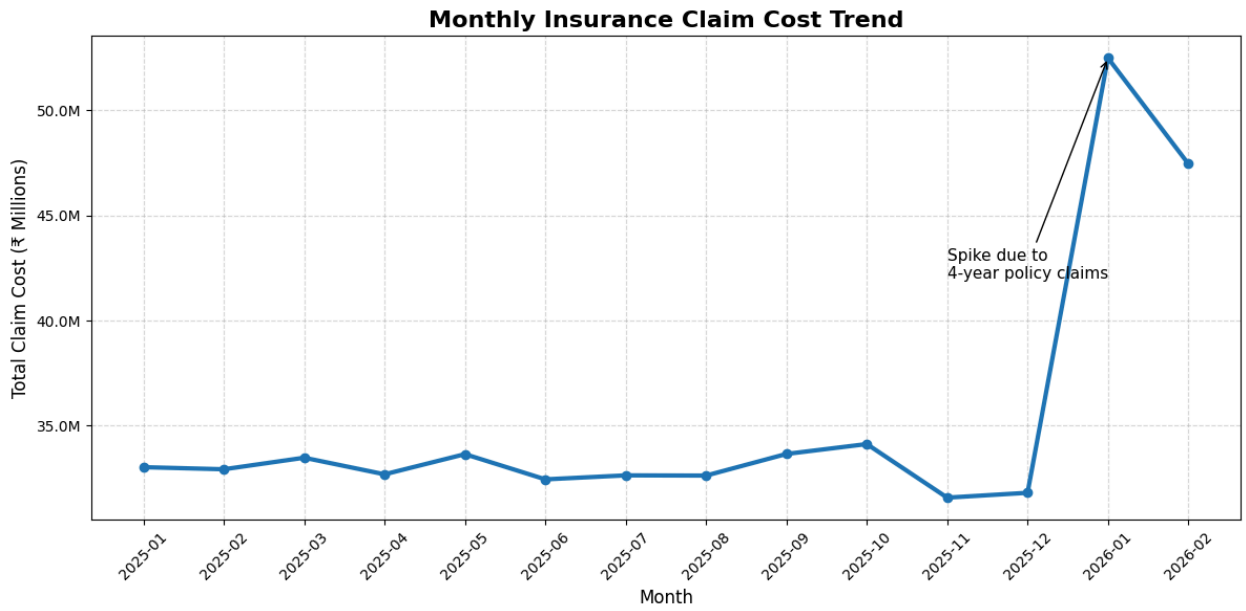
```
SELECT
strftime('%Y', Claim_Date) AS Claim_Year,
strftime('%m', Claim_Date) AS Claim_Month,
SUM(Claim_Amount) AS Total_Claim_Cost
FROM claims_data
GROUP BY
      Claim_Year, Claim_Month
ORDER BY
      Claim_Year, Claim_Month;
```

Result

	Claim_Year	Claim_Month	Total_Claim_Cost
0	2025	01	33020000.0
1	2025	02	32920000.0
2	2025	03	33470000.0
3	2025	04	32680000.0
4	2025	05	33640000.0
5	2025	06	32440000.0
6	2025	07	32630000.0
7	2025	08	32620000.0
8	2025	09	33650000.0
9	2025	10	34120000.0
10	2025	11	31570000.0
11	2025	12	31800000.0
12	2026	01	52500000.0
13	2026	02	47460000.0

Claims remain relatively stable during **2025** and increase significantly in **January and February 2026** due to additional claims from 4-year policies.

Graph



Part 3 — Claim Cost to Premium Ratio by Policy Tenure

SQL Query

```
SELECT
      p.Policy_Tenure,
      SUM(COALESCE(c.Claim_Amount,0)) AS Total_Claims,
      SUM(p.Premium) AS Total_Premium,
      ROUND(CAST(SUM(COALESCE(c.Claim_Amount,0)) AS FLOAT) /
SUM(p.Premium),2) AS Claim_Premium_Ratio
FROM
      policy_sales_data p
LEFT JOIN
      claims_data c
ON
      p.Vehicle_ID = c.Vehicle_ID
GROUP BY
      p.Policy_Tenure
ORDER BY
      p.Policy_Tenure;
```

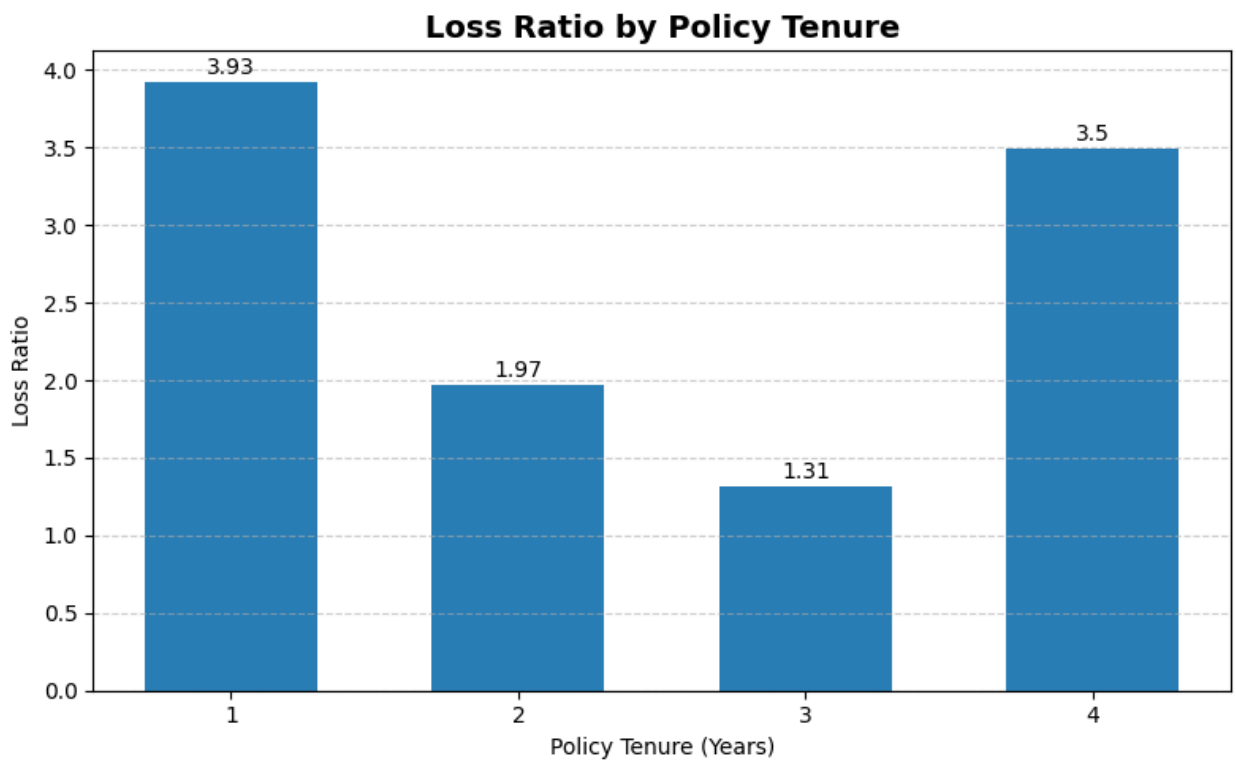
Result

	Policy_Tenure	Total_Claims	Total_Premium	Claim_Premium_Ratio
0	1	78290000.0	19937600	3.93
1	2	118040000.0	60021400	1.97
2	3	157810000.0	120165000	1.31
3	4	140380000.0	40157600	3.50

Interpretation

The **3-year policies appear to be the most profitable**, as they have the lowest loss ratio.

Graph



Part 4 — Claim Ratio by Policy Purchase Month

SQL Query

```
SELECT
      strftime('%m', p.Policy_Purchase_Date) AS Sale_Month,
      SUM(COALESCE(c.Claim_Amount,0)) AS Total_Claims,
      SUM(p.Premium) AS Total_Premium,
      ROUND(
      CAST(SUM(COALESCE(c.Claim_Amount,0)) AS FLOAT) / SUM(p.Premium),
      2
      ) AS Claim_Premium_Ratio
FROM
      policy_sales_data p
LEFT JOIN
      claims_data c
ON
      p.Vehicle_ID = c.Vehicle_ID
GROUP BY
      Sale_Month
ORDER BY
      Sale_Month;
```

Result

	Sale_Month	Total_Claims	Total_Premium	Claim_Premium_Ratio
0	01	41670000.0	20305900	2.05
1	02	41270000.0	19044400	2.17
2	03	41310000.0	20392800	2.03
3	04	41110000.0	19639900	2.09
4	05	42240000.0	20350200	2.08
5	06	40530000.0	19603700	2.07
6	07	40780000.0	20359900	2.00
7	08	40810000.0	20368800	2.00
8	09	41670000.0	19715100	2.11
9	10	42590000.0	20447600	2.08
10	11	39930000.0	19657600	2.03
11	12	40610000.0	20395700	1.99

Interpretation

Loss ratios remain relatively consistent across months, indicating no strong seasonal risk patterns in the simulated dataset.

Part 5 — Estimated Future Claim Liability

SQL Query

```
SELECT
      COUNT(*) * 10000 AS Estimated_Future_Claim_Liability
FROM
      policy_sales_data
```

```
WHERE
      Vehicle_ID NOT IN (
    SELECT Vehicle_ID FROM claims_data);
```

Result

	Estimated_Future_Claim_Liability
0	9509750000

Estimated Future Liability ≈ ₹9.5 Billion

Interpretation

This estimate assumes that every vehicle that has not yet filed a claim will eventually file one claim during the remaining policy tenure.

Part 6 — Earned Premium Calculation

Daily premium was calculated as:

Daily Premium = Total Premium / Total Policy Tenure Days

(i) Calculate the premium already earned by the company up to February 28, 2026.

```
SELECT
SUM(
(
MIN(julianday('2026-02-28'), julianday(Policy_End_Date))
-
julianday(Policy_Start_Date)
) /
(julianday(Policy_End_Date) - julianday(Policy_Start_Date))
* Premium
) AS Earned_Premium
FROM policy_sales_data
WHERE Policy_Start_Date <= '2026-02-28';
```

RESULT

	Earned_Premium
0	65869423.29

(ii)Estimate the premium expected to be earned monthly for the remaining policy period (assume 46 months remaining).

```
query = """
SELECT
SUM(Premium) AS Total_Premium,
SUM(julianday(Policy_End_Date) - julianday(Policy_Start_Date)) AS
Total_Tenure_Days
FROM policy_sales_data
"""

df = pd.read_sql_query(query, conn)

total_premium = df["Total_Premium"][0]
total_tenure_days = df["Total_Tenure_Days"][0]

# Step 1: Daily Premium
daily_premium = total_premium / total_tenure_days

# Step 2: Days elapsed until Feb 28 2026
query_days = """
SELECT
SUM(
CASE
WHEN Policy_Start_Date > '2026-02-28' THEN 0
WHEN Policy_End_Date < '2026-02-28'
THEN julianday(Policy_End_Date) - julianday(Policy_Start_Date)
ELSE julianday('2026-02-28') - julianday(Policy_Start_Date)
END
) AS Elapsed_Days
FROM policy_sales_data
"""

elapsed_days = pd.read_sql_query(query_days, conn)["Elapsed_Days"][0]
```

```
# Step 3: Earned Premium
earned_premium = daily_premium * elapsed_days

# Step 4: Remaining Premium
remaining_premium = total_premium - earned_premium

# Step 5: Monthly Remaining Premium (46 months)
monthly_remaining_premium = remaining_premium / 46

# Display results
print("Total Premium:", round(total_premium,2))
print("Total Policy Tenure Days:", round(total_tenure_days,2))
print("Daily Premium:", round(daily_premium,6))
print("Elapsed Policy Days:", round(elapsed_days,2))
print("Earned Premium (as of Feb 28 2026):", round(earned_premium,2))
print("Monthly Premium for Remaining 46 Months:",
round(monthly_remaining_premium,2))
```

Results

Total Premium: 240110800
Total Policy Tenure Days: 876404420.0
Daily Premium: 0.273973
Elapsed Policy Days: 240423395.0
Earned Premium (as of Feb 28 2026): 65869423.29
Monthly Premium for Remaining 46 Months: 3787856.02

Part 4 -Bonus Analysis

1. Identify which policy tenure appears most profitable and explain why

Metric Used

Loss Ratio = Total Claims / Total Premium

Lower loss ratio ⇒ **more profitable**, because a smaller portion of collected premium is paid out as claims.

Results from the Analysis

Policy Tenure	Total Premium	Total Claims	Loss Ratio
1 year	≈19.9M	≈78.3M	3.93
2 years	≈60.0M	≈118.0M	1.97
3 years	≈120.2M	≈157.8M	**1.31**
4 years	≈40.0M	≈140.4M	3.51

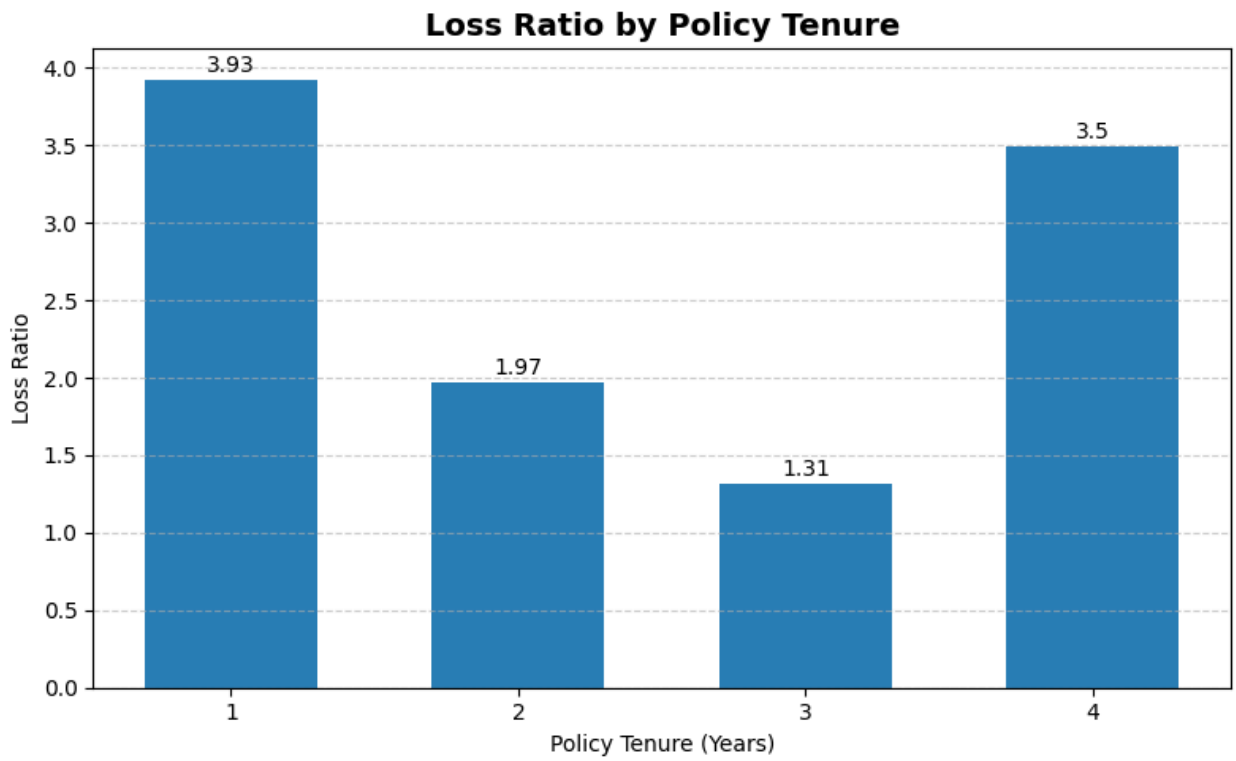
Most Profitable Policy Tenure

The 3-year policy tenure appears to be the most profitable.

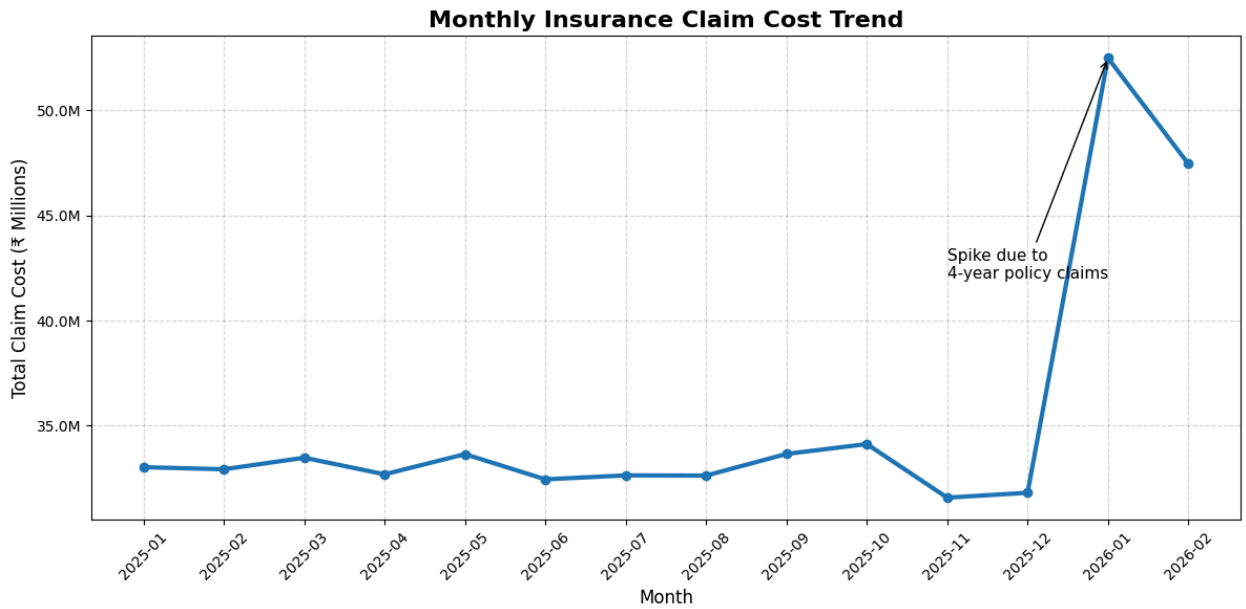
Reason

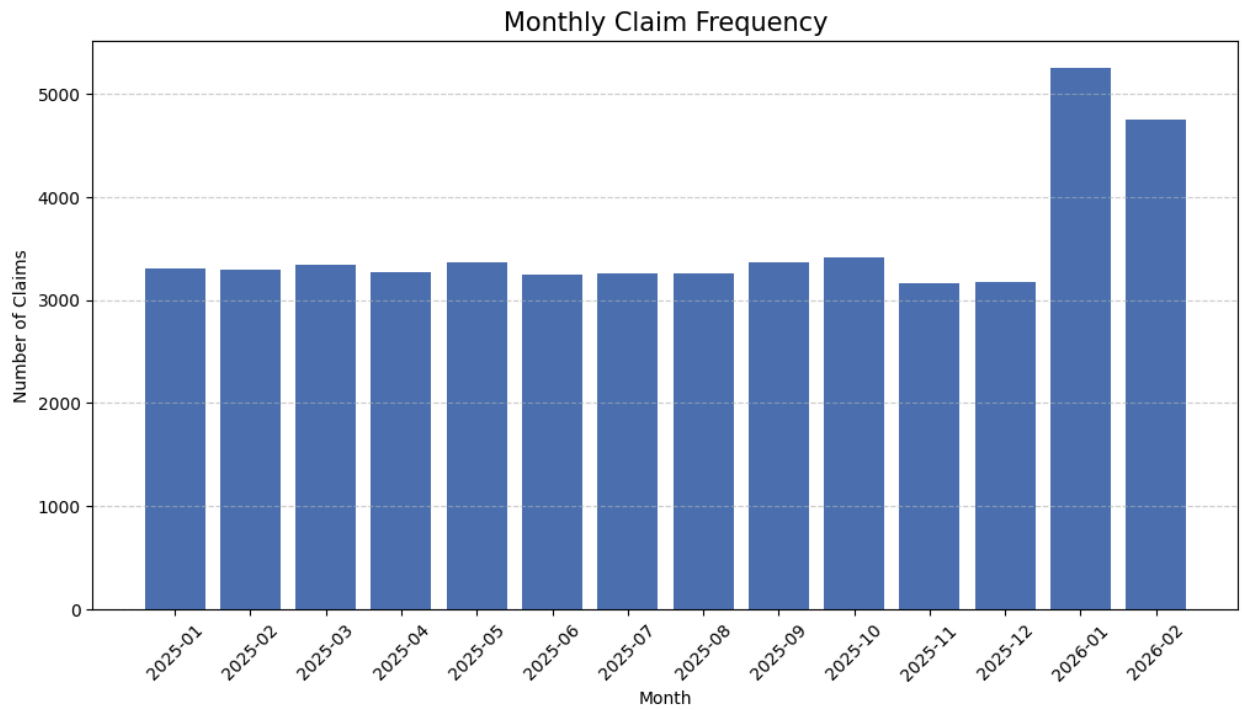
It has the **lowest loss ratio (≈1.31)** among all tenures.

Graph



2. Build a simple dashboard or visualization showing claim trends by month





3.Portfolio Loss Ratio

Loss Ratio = Total Claims / Total Premium

```
query = """
SELECT
```

```
SUM(Claim_Amount) AS Total_Claims
FROM claims_data
"""

claims = pd.read_sql_query(query, conn)["Total_Claims"][0]

query = """
SELECT
SUM(Premium) AS Total_Premium
FROM policy_sales_data
"""

premium = pd.read_sql_query(query, conn)["Total_Premium"][0]

loss_ratio = claims / premium

print("Total Claims:", claims)
print("Total Premium:", premium)
print("Portfolio Loss Ratio:", round(loss_ratio,2))
```

Result:

Total Claims: 494520000.0
Total Premium: 240110800
Portfolio Loss Ratio: **2.06**

This indicates that total claims exceed premium revenue in the simulated dataset.

IMPORTANT-**This occurs because the simulated claim amount (₹10,000) is significantly larger than the policy premiums (₹100–₹400).**

Impact of 5% Increase in Claim Frequency

```
claim_count_query = """
SELECT COUNT(*) AS Total_Claims
FROM claims_data
"""
```

```
claims_count = pd.read_sql_query(claim_count_query,
conn)["Total_Claims"][0]

new_claims = claims_count * 1.05

claim_amount = 10000

future_claim_cost = new_claims * claim_amount

new_loss_ratio = future_claim_cost / premium

print("Current Claims:", claims_count)
print("Claims after 5% increase:", round(new_claims))
print("Future Claim Cost:", round(future_claim_cost))
print("New Loss Ratio:", round(new_loss_ratio,2))
```

If claim frequency increases by **5% annually**:

Current Claims: 49452

Claims after 5% increase: 51925

Future Claim Cost: 519246000

New Loss Ratio: **2.16**

Scenario	Loss Ratio
Current	**2.06**
After Increase	**2.16**

This would further reduce portfolio profitability.

Key Insights

1. **3-year policies appear to be the most profitable segment of the portfolio.**
   Among all policy tenures, 3-year policies have the lowest claim-to-premium ratio. This suggests that the premium collected from these policies is relatively better balanced against the claims generated.

2. **Claims remain relatively stable during 2025 but increase sharply in early 2026.**
   Monthly claim costs during 2025 show a consistent pattern, indicating evenly distributed risk. However, a noticeable spike occurs in January and February 2026 due to the rule that 10% of 4-year policies file claims during this period.

3. **Policy tenure has a strong impact on portfolio risk.**
   While longer tenures generate higher premium revenue, they also increase the exposure period for claims. This effect is evident in the higher claim costs associated with 4-year policies.

4. **The portfolio currently shows a high loss ratio.**
   The estimated portfolio loss ratio exceeds 1, meaning that total claim costs are greater than the premium collected under the simulation assumptions. This highlights the importance of proper pricing and risk management in insurance portfolios.

5. **The portfolio remains exposed to significant future liabilities.**
   A large number of vehicles have not yet filed claims, which implies that substantial potential claim costs may still arise during the remaining policy tenure.

6. **Profitability is highly sensitive to claim frequency.**
   Even a modest increase in claim frequency (e.g., 5% annually) leads to a noticeable increase in the loss ratio, demonstrating how small changes in claim behavior can significantly affect insurer profitability.